

Searching in Tries, Compressed Tries, and Suffix Trees

1 Searching in a Standard Trie

To search for a word P :

1. Start at the root.
2. Read the next character of P .
3. Follow the matching child.
4. Repeat until all characters are consumed.
5. Confirm that the final node marks the end of a stored word.

SearchTrie

```
SearchTrie(root, word):  
    node = root  
  
    for each character c in word:  
        if node has no child labelled c:  
            return false  
  
        node = child labelled c  
  
    return node marks a complete word
```

2 Prefix Search

Prefix search follows the same path, but the last node does not need to be a complete-word marker.

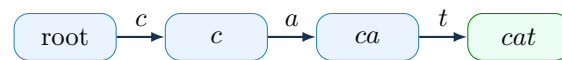
HasPrefix

```
HasPrefix(root, prefix):  
    node = root  
  
    for each character c in prefix:  
        if node has no child labelled c:  
            return false  
  
        node = child labelled c  
  
    return true
```

Difference

Word search asks: “Does this exact word exist?” Prefix search asks: “Does any stored word begin with this prefix?”

3 Standard-Trie Search Trace



query cat succeeds if final node is terminal

4 Searching in a Compressed Trie

In a compressed trie, an edge may contain a substring. The search must compare the query against the entire edge label.

SearchCompressedTrie

```
SearchCompressedTrie(root, word):
    node = root
    position = 0

    while position < length(word):
        choose an outgoing edge whose first character
        matches word[position]

        if no such edge exists:
            return false

        compare the edge label with the corresponding
        substring of word

        if the full edge label does not match:
            return false

        advance position by length(edge label)
        node = edge destination

    return final position represents a stored word
```



compare whole edge labels, not only one character

5 Searching in a Suffix Tree

To search for a pattern P :

1. Start at the root.
2. Choose an edge whose first character matches the next character of P .
3. Compare P against the edge label.
4. Continue until all of P is matched or a mismatch occurs.

If P is fully matched, then the descendant leaves identify where P occurs in the original text.

6 Comparison

Structure	Search rule
Standard trie	Compare one character per edge.
Prefix search	Same as word search, but no terminal mark required at the final node.
Compressed trie	Compare an entire substring edge label before moving on.
Suffix tree	Match a pattern across edge labels; descendant leaves give occurrences.

7 Search-Engine Indexing Idea

Inverted Index

For each keyword, store the list of document positions where it appears. A compressed trie can store the keywords, and each leaf points to that keyword's occurrence list.

Query type	How it is answered
One keyword	Search the keyword in the trie, then return its occurrence list.
Several keywords	Retrieve all occurrence lists and intersect them.
Prefix query	Walk to the prefix node and collect descendant keyword leaves.

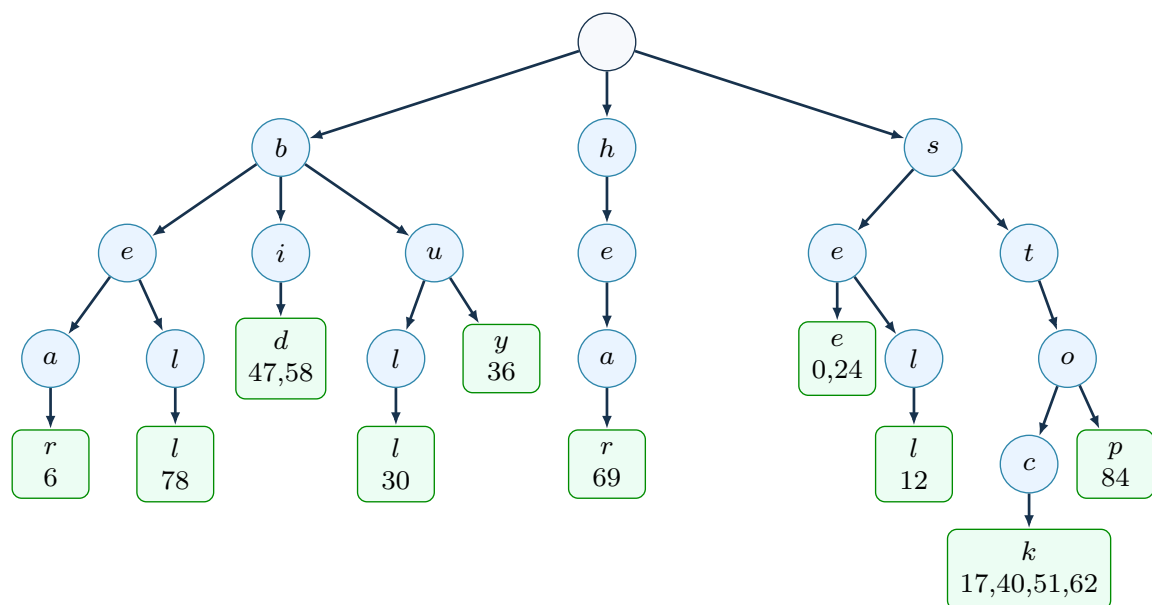


Diagram Meaning

The path spells the word; the leaf stores where that word begins. For example, the path stock ends with the occurrence list 17, 40, 51, 62.

Why store lists outside the trie?

Occurrence lists can be very large, so the trie stores only the dictionary structure and pointers to lists. The lists themselves are stored separately.